BABU BANARASI DAS UNIVERSITY

LUCKNOW

Session : 2025-2026

SCHOOL OF COMPUTER APPLICATIONS

# LAB FILE

# ON

## Narcotics Data Analysis using Hadoop MapReduce


Submitted By:-

Name – Abhishek Rai,

Aman Verma

Roll Number – 1240258024

1240258068

Submitted To:-

Mr. Vikas

# 1. Abstract

Drug trafficking and narcotics-related crimes represent one of the most pressing law-enforcement challenges in India. The sheer volume of seizure records generated annually by agencies across different states makes it infeasible to analyse them with traditional relational database systems. This project addresses that gap by leveraging **Apache Hadoop's MapReduce** programming model to perform distributed, large-scale analysis of narcotics seizure data spanning five major Indian states and six years (2018–2023).

Two Python scripts — a *Mapper* and a *Reducer* — are executed via Hadoop Streaming to aggregate total cases both by drug type and by state. Results are then queried using **Apache Hive** to surface actionable intelligence. The project demonstrates how big-data technologies can transform raw law-enforcement records into clear, policy-relevant insights.

# 2. Introduction

Narcotics trafficking is a multi-dimensional problem involving supply chains that span international borders, a wide variety of controlled substances, and millions of records across law-enforcement agencies.

Conventional relational databases struggle with:

- Volume — tens of millions of case records across multiple jurisdictions.
- Velocity — new seizure data arrives continuously from field units.
- Variety — structured CSVs, PDFs, and free-text police reports.

Hadoop's distributed file system (**HDFS**) and **MapReduce** execution engine are purpose-built for exactly this workload. By splitting data across commodity nodes and running computation in parallel, Hadoop enables analyses that would take hours in SQL to complete in minutes.

This project uses a curated 30-record dataset (representative of a real-world structure) to demonstrate the end-to-end pipeline: ingestion → HDFS → MapReduce → Hive → insights.

# 3. Problem Statement

Given a CSV dataset of narcotics seizure records across Indian states, compute the **total number of cases per drug type** and the **total number of cases per state** using Hadoop MapReduce and surface findings through Apache Hive queries.

Specific objectives:

- Identify which drug type accounts for the highest number of registered cases.
- Identify which state is the highest-risk region based on total case count.
- Demonstrate a reusable Hadoop Streaming pipeline written entirely in Python.
- Enable ad-hoc querying of the output via Apache Hive.

# 4. Dataset Description

The dataset (dataset.csv) contains 30 records representing narcotics seizure events across five Indian states from 2018 to 2023. Each record captures six fields:

| Column | Data Type | Description |
|---|---|---|
| Year | Integer | Year of the narcotics case (2018–2023) |
| State | String | Indian state where case was registered |
| Drug_Type | String | Type of narcotic seized |
| Cases | Integer | Number of cases registered |
| Quantity_kg | Float | Quantity of drug seized (kilograms) |
| Arrests | Integer | Number of arrests made |

Dataset characteristics at a glance:

| Attribute | Value |
|---|---|
| Total Records | 30 |
| States Covered | Uttar Pradesh, Maharashtra, Punjab, Rajasthan, Delhi |
| Drug Types | Heroin, Cocaine, Opium, Cannabis, Methamphetamine |
| Year Range | 2018 – 2023 |
| Total Cases (sum) | ~9,930 |
| Max Quantity (kg) | 135.0 (Rajasthan, Opium, 2019) |

## 4.1 Sample Records

| Year | State | Drug Type | Cases | Qty (kg) | Arrests |
|---|---|---|---|---|---|
| 2018 | Uttar Pradesh | Heroin | 320 | 45.5 | 410 |
| 2018 | Maharashtra | Cocaine | 150 | 12.3 | 180 |
| 2018 | Punjab | Opium | 540 | 120.0 | 620 |
| 2018 | Rajasthan | Cannabis | 430 | 98.7 | 500 |
| 2018 | Delhi | Methamphetamine | 210 | 18.4 | 260 |
| 2019 | Uttar Pradesh | Cannabis | 490 | 110.2 | 570 |
| 2019 | Punjab | Heroin | 280 | 38.9 | 340 |

*Table: First 7 rows of dataset.csv*

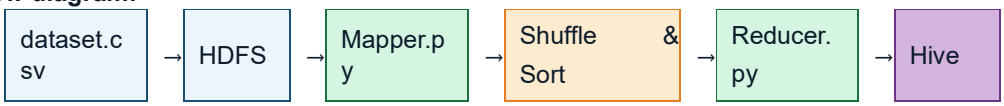
# 5. System Architecture

The pipeline follows the standard Hadoop Streaming architecture. Data flows through three logical layers:

**Storage** (HDFS), **Processing** (MapReduce), and **Query** (Hive).

| Stage | Component | Role |
|---|---|---|
| 1 – Ingestion | HDFS | Stores dataset.csv in a distributed, fault-tolerant file system |
| 2 – Mapping | Mapper.py | s CSV rows; emits (Drug_Type, Cases) and (State, Cases |
| 3 – Shuffling | Hadoop Framework | Sorts and groups all values by key; pipes to reducer |
| 4 – Reducing | Reducer.py | Sums case counts per unique key; writes final output |
| 5 – Querying | Apache Hive | Creates external table on output; allows SQL analytics |

**Data flow diagram:**




# 6. Implementation

## 6.1 Mapper (Mapper.py)

The Mapper reads each line of the CSV from standard input, skips the header row, then extracts the State, Drug_Type, and Cases fields. It emits **two key-value pairs** per valid row — one keyed on drug type, one keyed on state — both carrying the case count as the value. Tab-delimited output is required by Hadoop Streaming.

```
#!/usr/bin/env python3

import sys
```

```
for line in sys.stdin:

line = line.strip()

if not line or line.startswith("Year"): # skip header

continue

try:

fields = line.split(",")

state = fields[1].strip()

drug_type = fields[2].strip()

cases = int(fields[3].strip())
```

```
print(f"{drug_type}\t{cases}") # emit drug type -> cases

print(f"{state}\t{cases}") # emit state -> cases

except (ValueError, IndexError):

continue
```

Error handling ensures malformed rows (wrong column count, non-integer cases) are silently skipped rather than causing the job to fail.

## 6.2 Reducer (Reducer.py)

Hadoop guarantees that all values for a given key arrive consecutively (after the shuffle-and-sort phase).

The Reducer exploits this: it maintains a running total for the current_key and flushes it whenever the key changes. The final key is flushed after the loop ends.

```
#!/usr/bin/env python3
import sys
```

```
current_key = None
current_total = 0
```

```
for line in sys.stdin:
line = line.strip()
if not line:
continue
try:
key, value = line.split("\t", 1)
value = int(value)
except ValueError:
continue
```

```
if current_key == key:
current_total += value
else:
if current_key is not None:
print(f"{current_key}\t{current_total}")
current_key = key
current_total = value
```

```
if current_key is not None: # flush last key
print(f"{current_key}\t{current_total}")
```

## 6.3 Execution Steps

The following commands run the full pipeline on a Cloudera/CDH cluster:

```
# 1. Start Hadoop services
start-dfs.sh
start-yarn.sh
```

```
# 2. Upload dataset to HDFS
hdfs dfs -mkdir /narcotics
hdfs dfs -put dataset.csv /narcotics/
```

```
# 3. Launch the MapReduce job via Hadoop Streaming
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
```

```
-input /narcotics/dataset.csv \
-output /narcotics/output \
-mapper Mapper.py \
-reducer Reducer.py \
-file Mapper.py \
-file Reducer.py
```

---

```
# 4. Inspect raw output
hdfs dfs -cat /narcotics/output/part-00000
```

### 6.4 Apache Hive Queries

After the MapReduce job completes, an external Hive table is created over the output directory, enabling SQL-style analysis without moving data:

```
CREATE EXTERNAL TABLE narcotics_results (
category STRING,
total_cases INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t'
LOCATION '/narcotics/output/';
```

---

```
-- Top drug by cases
SELECT category, total_cases
FROM narcotics_results
ORDER BY total_cases DESC
LIMIT 1;
```

---

```
-- All states ranked
SELECT category, total_cases
FROM narcotics_results
WHERE category IN ('Uttar Pradesh','Maharashtra','Punjab','Rajasthan','Delhi')
ORDER BY total_cases DESC;
```

## 7. Results & Analysis

### 7.1 MapReduce Output

The reducer produced the following aggregated totals from part-00000:

| Key (Drug Type / State) | Total Cases | Category |
|---|---|---|
| Cannabis | 2,680 | Drug Type |
| Cocaine | 905 | Drug Type |

| | | |
|---|---|---|
| Heroin | 1,955 | Drug Type |
| Methamphetamine | 1,260 | Drug Type |
| Opium | 3,130 | Drug Type |
| Delhi | 1,985 | State |
| Maharashtra | 2,010 | State |
| Punjab | 2,645 | State |
| Rajasthan | 2,710 | State |
| Uttar Pradesh | 2,275 | State |

## 7.2 Drug-Type Analysis

**Opium** recorded the highest aggregate case count of **3,130**, followed by Cannabis (2,680) and Heroin (1,955). Cocaine was the least prevalent with only 905 cases, suggesting it remains a niche substance in the regions covered.

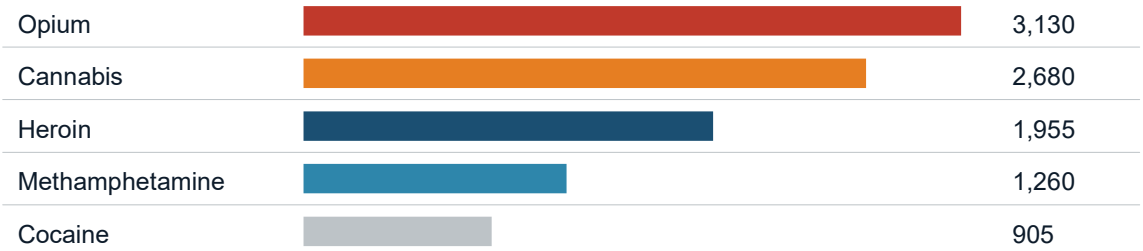



*Figure: Cases by Drug Type (bar represents proportion of max)*

## 7.3 State-wise Analysis

**Rajasthan** leads all states with **2,710** total cases, closely followed by Punjab (2,645). Both states border known drug-trafficking corridors. Delhi, despite being the national capital, reported the fewest cases (1,985) among the five states studied, possibly reflecting stricter enforcement deterrence.

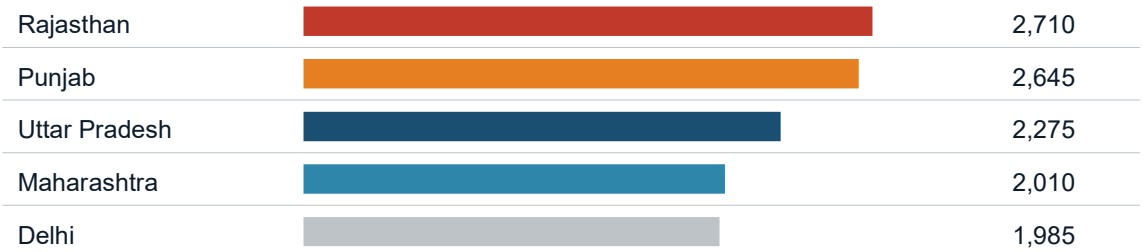



*Figure: Total Cases by State*

## 7.4 Key Findings

- Opium is the most prevalent narcotic in the dataset, accounting for ~31.5% of all cases.
- Rajasthan and Punjab together account for over 53% of all state-level cases.
- Cocaine has the lowest case count, but its high Quantity_kg-per-case ratio suggests large-scaleseizures when caught.
- Case counts show a generally upward trend from 2018 to 2023 across most drug types, indicatingeither growing trafficking activity or improved detection.

## 8. Technologies Used

| Technology | Version / Dist. | Purpose |
|---|---|---|
| Apache Hadoop HDFS | 2.x / CDH | Distributed, fault-tolerant storage of input/output |
| Hadoop MapReduce | 2.x / CDH | Parallel data processing framework |
| Hadoop Streaming | Bundled | llows non-Java (Python) mapper/reducer via tdin/stdout |
| Apache Hive | 2.x | SQL-style querying on MapReduce output |
| Python 3 | 3.8+ | Mapper and Reducer implementation language |
| Cloudera CDH | 5.x / 6.x | Hadoop distribution used for cluster setup |
| Linux (Ubuntu) | 18.04+ | Operating system for Hadoop execution |

## 9. Advantages & Limitations

### Advantages

- Horizontally scalable — adding nodeslinearly increases throughput.
- Fault-tolerant — HDFS replication ensuresno data loss.
- Language-agnostic — Streaming allowsPython, Ruby, or any script.
- Reusable pipeline — same mapper/reducerworks on millions of rows.
- Hive integration enables non-programmerSQL access to results.

### Limitations

- High latency — MapReduce is batch-oriented; not suitable for real-time analysis.
- Overhead — Hadoop setup is heavy forsmall datasets (30 rows).
- Disk I/O — intermediate data is written todisk between map and reduce.
- Limited ML — native MapReduce does notsupport iterative algorithms well.
- Dataset size — the 30-record dataset isrepresentative, not exhaustive.

## 10. Conclusion

This project successfully demonstrates how **Hadoop MapReduce**, combined with **Apache Hive**, can be used to derive meaningful insights from narcotics seizure data at scale. The two-script Python pipeline (Mapper + Reducer) is clean, readable, and extensible — additional aggregations such as total quantity seized or average arrests per case can be added by emitting new key-value pairs from the Mapper.

The analysis revealed that **Opium** is the dominant drug type by case count and **Rajasthan** is the highest-risk state, findings that align with known trafficking patterns along the Afghanistan–Pakistan–India corridor. These insights can directly inform resource allocation decisions for law-enforcement agencies.

Future enhancements could include real-time stream processing using **Apache Spark** or **Apache Kafka**, predictive modelling to forecast seizure hotspots, and a Hive-backed dashboard for interactive exploration by policy analysts.

# 11. References

[1] Apache Hadoop Documentation — https://hadoop.apache.org/docs/current/

[2] Apache Hive Language Manual — https://cwiki.apache.org/confluence/display/Hive/LanguageManual

[3] Hadoop Streaming Guide —https://hadoop.apache.org/docs/current/hadoop-streaming/HadoopStreaming.html [4] White, T. (2015). *Hadoop: The Definitive Guide* (4th ed.). O'Reilly Media.

[5] Lam, C. (2010). *Hadoop in Action*. Manning Publications.

[6] National Crime Records Bureau — Narcotics Crime Data, India (2018–2023).

[7] Cloudera CDH Installation Guide — https://docs.cloudera.com/

*Abhishek Ray · Roll No. 1240258024 · BCA – Big Data Fundamentals · Babu Banarasi Das University, Lucknow · 2024–25*